

Task 1.

a) Explain what the C programming language can be used for?

C is a general-purpose programming language. It is mainly used in situations where performance and low-level control are required. As a compiled language, C is capable of being significantly faster than interpreted languages such as Python or JavaScript. It is commonly used for operating systems, embedded systems, and compilers (Kernighan & Ritchie, 1988; Britannica Editors, 2026). Since C allows direct manipulation of memory and data, it is well suited for writing efficient programs that interact closely with hardware. In conclusion, C enables the development of software that other software and hardware depend on.

b) Who is Richard Stallman and what is he known for in the field of Information Technology?

Richard M. Stallman is an American programmer and free software activist. He is best known for starting the GNU Project in 1983, aiming to create a free Unix-like operating system (Stallman, 1983). He is also known as a central figure in the free software movement, which promotes the idea that software should be freely available to use, modify, and distribute. In addition, he developed GNU Emacs. He is widely regarded as one of the key figures whose work laid the foundation for modern GNU/Linux systems. Stallman initiated the GNU Project, while Linus Torvalds later developed the Linux kernel, which together form the basis of GNU/Linux systems.

c) Explain what the sudo command is used for on a Linux-based system?

The sudo command stands for super user do (Sudo Project, 2025). The Linux manual page describes sudo as allowing a permitted user to execute a command as the superuser or another user, depending on the configured security policy (Kerrisk, 2025). The Sudo Project manual also explains that sudo permissions are controlled by a security policy, commonly the sudoers policy (Sudo Project, 2025) this one can access in the sudoers file /etc/sudoers.

References:

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language*. (2nd ed.). Prentice Hall, Pearson.

Britannica Editors (2026, March 4). C. Encyclopedia Britannica. <https://www.britannica.com/technology/C-computer-programming-language>

Kerrisk, M. (2025). *sudo(8): Linux manual page*. Linux man-pages project. <https://man7.org/linux/man-pages/man8/sudo.8.html>

Stallman, R. M. (1983). *Initial announcement*. GNU Project. <https://www.gnu.org/gnu/initial-announcement.en.html>

Sudo Project. (2025). *Sudo manual*. Sudo Project. <https://www.sudo.ws/about/intro/>

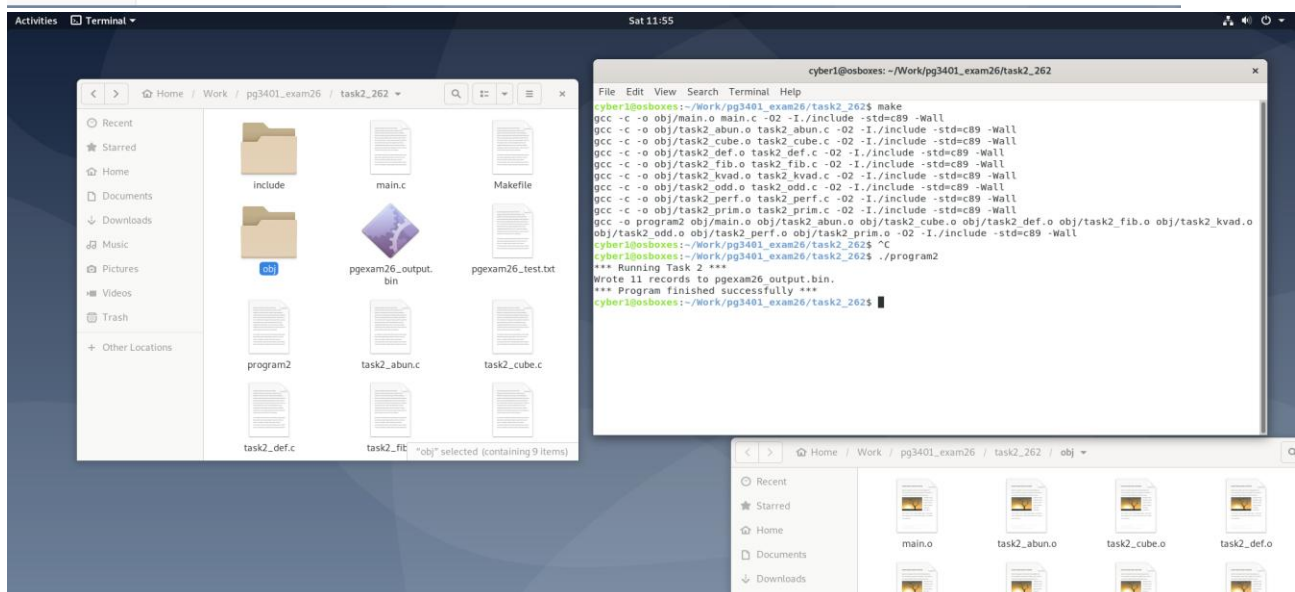
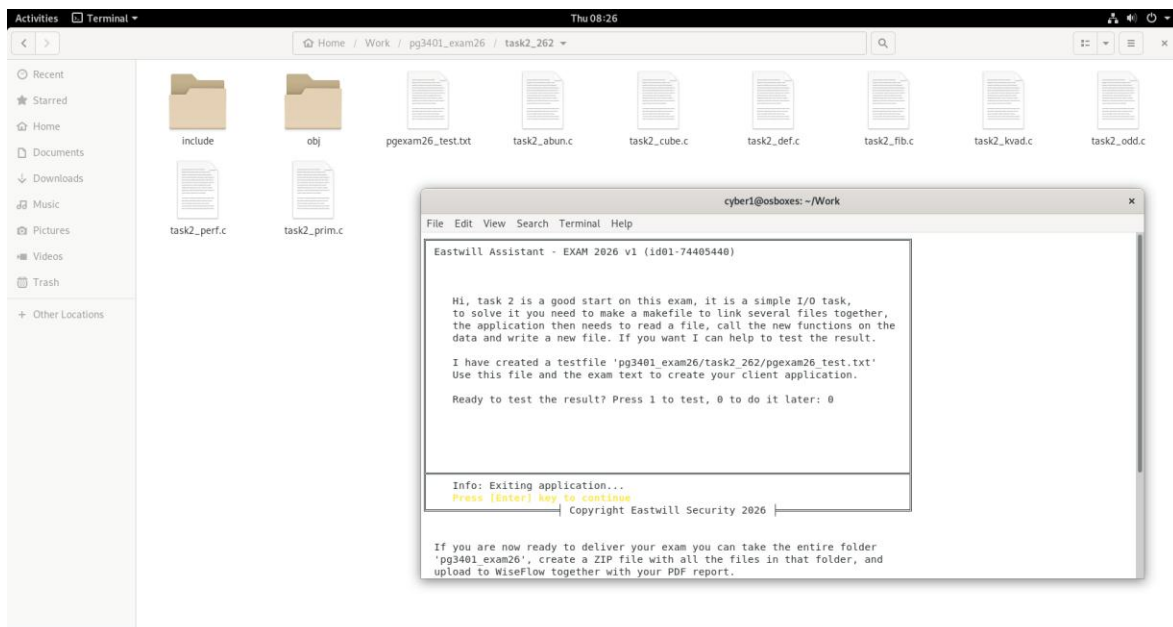
Task 2. Documentation.

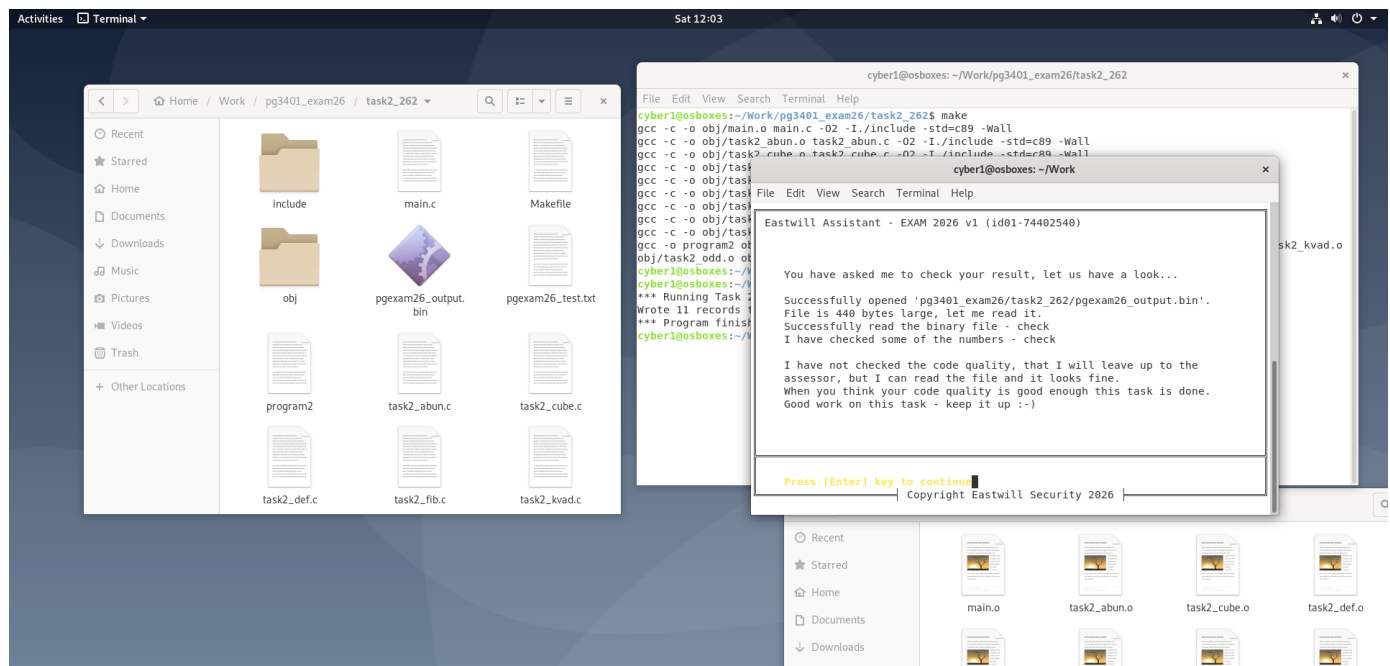
Considerations & execution choices: Makefile: I chose to add std c89 and Wall flags. To help me keep within C89 and for the compiler to give me warnings.

Provided source files used the libraries: <math.h> a standard of c89 (p. 250, b4, of the C course book).

Yet some of the functions used here were still of later 99 standard: cbrt(). Then <stdbool.h> is not standard, it's part of c99. I changed the function files somewhat, away from bool to int and output where needed: 1 true, 0 false. Only thing that needed more than minor rework was the cube file that used cbrt(). At that point I figured I could change the kvad.c also (not use sqrt()). Then I made 8 baby header files for them.

Main.c initially I implemented using a stack-allocated struct, seemed fine for the specific case. I pondered a bit on it and changed to dynamic with malloc/free. Thinking then: use dynamic, given the context of this exam. But it is not strictly needed here.





Issues: Misread, first image slipped; so restarted EWA for this. Did skim over, but middle bit slipped for me as the first photo was mentioned second in the sentence and yea the third image came right after.

Testing of larger numbers than we had here could be an issue. Quadratic x^2 and cube formula x^3 can naturally fast grow rather large and overflow. Use of long int could mitigate this (until a point). Beyond that one would have to implement BigInt for C, using a carry mechanism to handle the overflow. Did not implement this. But I had recently seen how it could be done for C by Tsoding (live on twitch). Full video: <https://www.youtube.com/watch?v=tbPOFYwL7Ss&t=364s>

Finally running EWA again for verification check, overwrote the 8 files, but anticipated had backed up all.

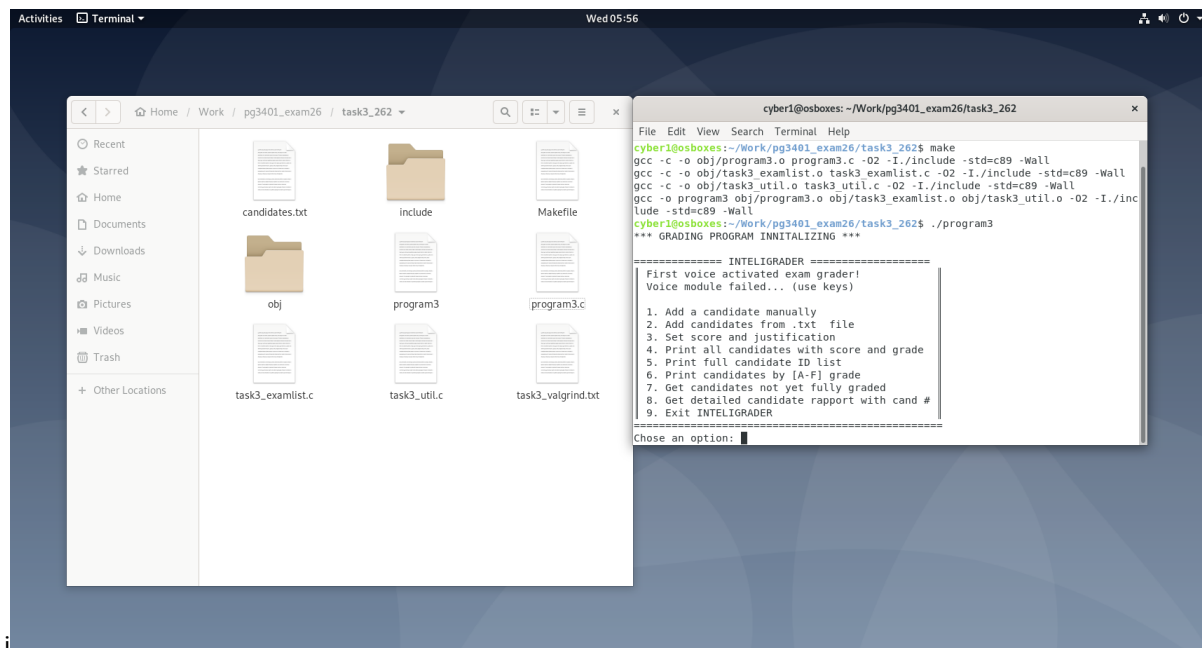
Task 3. Documentation.

Considerations & execution: Ended up doing this task last. Knew I needed more time on the later tasks, and was a bit paralyzed here (had a lot of ideas, but not quite sure what to go with). Went with very basic “INTEIGRADER” (voice activated grader, well ofc not really). But I did attempt to make it look a little neat. Borrowed sidebars “||” from EWA. I also used [Enter] to continue, like EWA uses. That was maybe not a very bright idea, it became somewhat tedious during testing.

I split the file in program3.c for main: main/menu handling, task3_util.c with header for input and helper menu functions, and task3_examlist.c with header for linked list and grading logic. I tried to make it so none of one source files grew too large: rather split/ delegate better. I think I managed this better here.

Makefile: Just went with std 89, Wall and -g. For debugging I used the compiler and the program itself.

I first made the program with options for adding a candidate manually, printing all candidates, and exiting normally. This tested the double linked list with head/tail pointers and cleanup on exit. Compiled and tested. Secondly, I added the option to set score, justification, and print all candidates with score/grade. I had a small snag here with saving the scores correctly. Then I implemented reading candidates from a file and later added the remaining print/report options bit by bit. Lastly I decided to change how the manual grading was done, placed a new loop here and made it so one does not have to enter from the menu for each task assessment. Ran valgrind at the end and saved output to task3_valgrind.txt



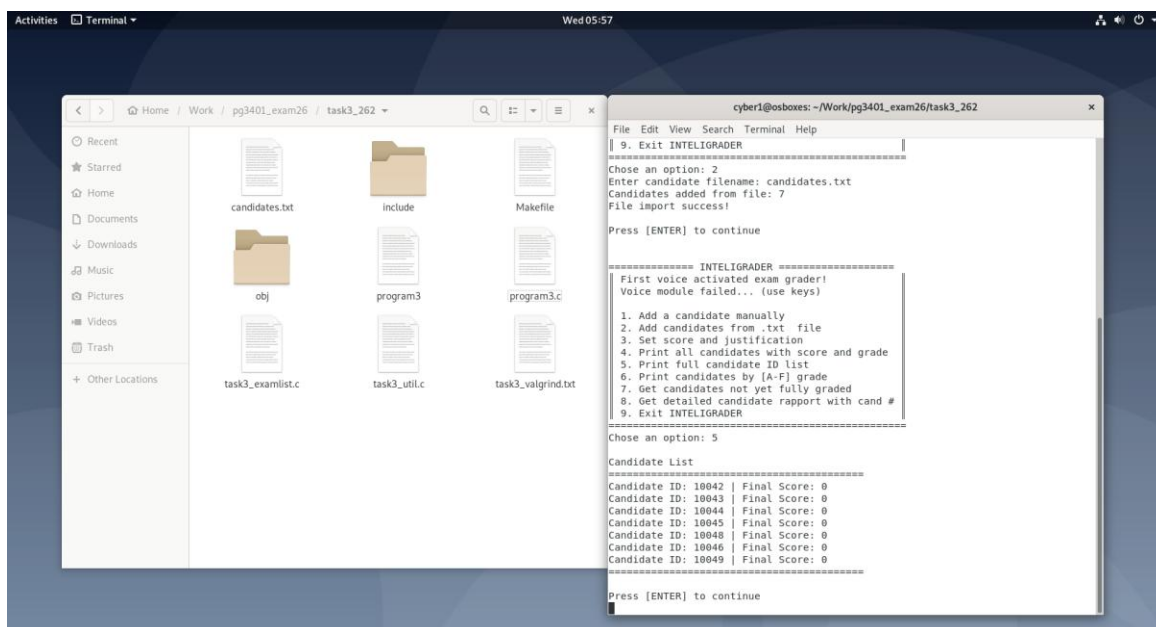
Issues: Notably grading / scores were an issue for a while I had accidentally written to aTaskPoints twice, so the real score was overwritten, score became the task number:

```
pCandidate->aTaskPoints[iTaskIndex] = iPoints;
```

```
pCandidate->aTaskPoints[iTaskIndex] = 1;
```

Instead, then of aTaskGraded on the second line (marked it YXXX in examlist.c). Compiler did not complain there about that one & no valgrind issue, naturally as it is still sound. Just wrong.

In retrospect my first solution to manually assess with score and justification was rather poor, had it so you could pick any task. And after inputting student Id- task number – score – justification: would take one back to the menu. So, to fully grade one student 30 inputs (with the menu choice, 60 if we count enter), not exactly very efficient.



Task 4. Documentation.

Considerations & execution part 1: Included the relevant source files and header file to the Makefile. Flags: added-pthread from the start given it is a thread task. To assist debugging, I also added in a -g flag.

Threads.c, I changed partial Bool logic to using 1/0. Removed global variables using a shared struct “THREADDATA” to pass on data to the threads. Removed various issues (*) set wrong placed wrong, and several ones I also introduced.

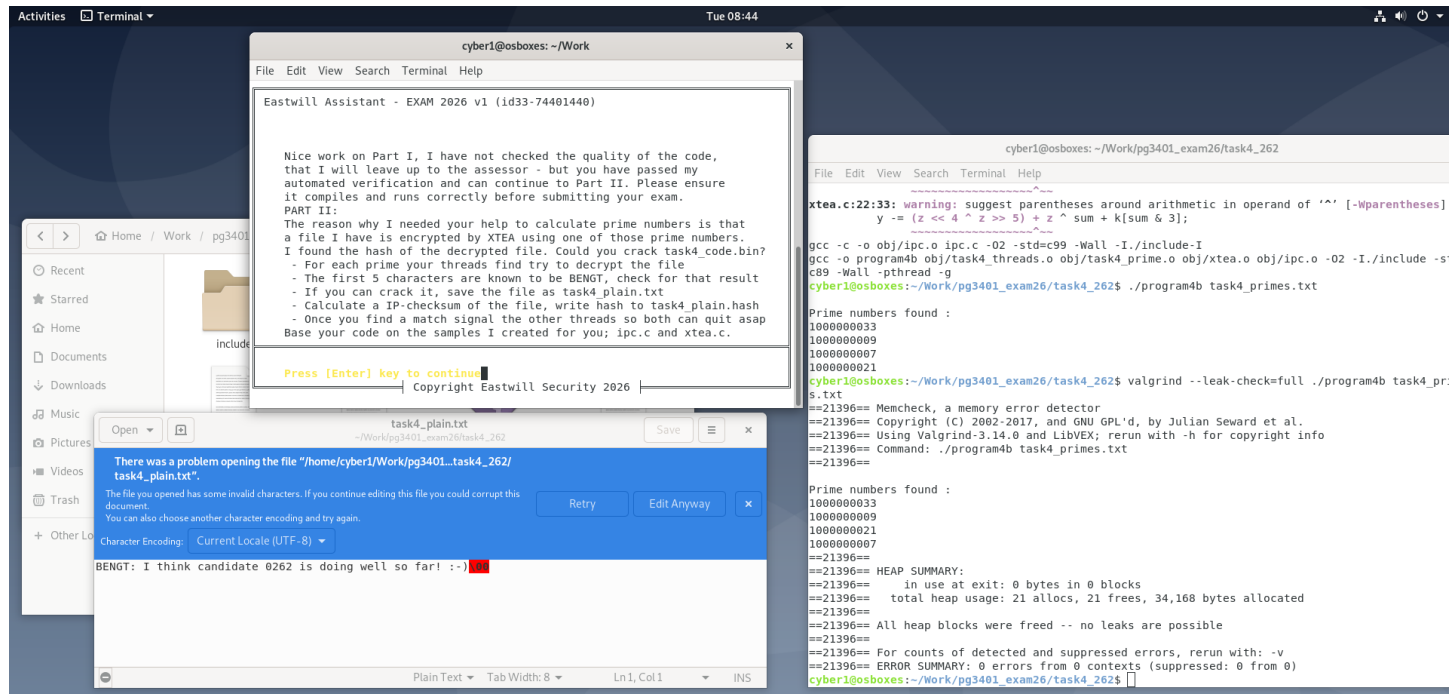
Mutexes I Initialized in main before creating the threads, added locks - unlocks around the creation of nodes “listMutex”, & around the read function “fileMutex”(fscanf read-write). Used destroy Mutexes on error paths and the normal exit. Did make sure to have join prior to this (page 51 lec 8). Also made sure I did not use join on the first create thread error: second worker thread would not have been spawned in by then. (Sucks to wait on someone not coming. And yes, I did once invite to a party of 1).

```
cyber1@osboxes: ~/Work/pg3401_exam26/task4_262
File Edit View Search Terminal Help
gcc -c -o obj/task4_threads.o task4_threads.c -O2 -I./include -std=c89 -Wall -pthread -g
gcc -c -o obj/task4_prime.o task4_prime.c -O2 -I./include -std=c89 -Wall -pthread -g
gcc -o program4a obj/task4_threads.o obj/task4_prime.o -O2 -I./include -std=c89 -Wall -pthread -g
cyber1@osboxes:~/Work/pg3401_exam26/task4_262$ valgrind --leak-check=full ./program4a task4_prime
s.txt
==17403== Memcheck, a memory error detector
==17403== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==17403== Using Valgrind-3.14.0 and LibVEX; rerun with -h for copyright info
==17403== Command: ./program4a task4_primes.txt
==17403==
Prime numbers found :
1000000033
1000000009
1000000021
1000000007
==17403==
==17403== HEAP SUMMARY:
==17403==   in use at exit: 0 bytes in 0 blocks
==17403==   total heap usage: 9 allocs, 9 frees, 6,280 bytes allocated
==17403==
==17403== All heap blocks were freed -- no leaks are possible
==17403==
==17403== For counts of detected and suppressed errors, rerun with: -v
==17403== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
cyber1@osboxes:~/Work/pg3401_exam26/task4_262$ make clean
rm -f obj/*.o *~ core ./include/*~
cyber1@osboxes:~/Work/pg3401_exam26/task4_262$ make
gcc -c -o obj/task4_threads.o task4_threads.c -O2 -I./include -std=c89 -Wall -pthread -g
gcc -c -o obj/task4_prime.o task4_prime.c -O2 -I./include -std=c89 -Wall -pthread -g
gcc -o program4a obj/task4_threads.o obj/task4_prime.o -O2 -I./include -std=c89 -Wall -pthread -g
cyber1@osboxes:~/Work/pg3401_exam26/task4_262$ ./program4a task4_primes.txt
Prime numbers found :
1000000009
1000000033
1000000007
1000000021
cyber1@osboxes:~/Work/pg3401_exam26/task4_262$
```

Part 2: Cool! (expected “BENGT ER KUL”). Saw on the “Kristiania linjeforening” discord a student “nour00” mention one could compile 1 file in C99 with an exception (don’t know this student but I also recall a young lady mentioned this in class). I chose to try this route, partly to test the method and learn something different. Got a missing header error, adding <stddef.h> to ipc.c and threads.c. First attempts using c99 for ipc.c seemed ok but this only compiled ipc.o. Moved the rule for beneath the program4b: \$(OBJS); then it compiled cleanly. With only compiler warnings being suggestions of changes (the lack of parenthesis use in the function tean). I would say it is still minor changes to the makefile: as in one could just set the overall flag to c99 or remove std flags entirely and it should compile without issues.

Pt2: Threads.c, saved old as task4_pt1_ for Makefile & threads.c. Endian coding and decoding ref linked xtea.pdf. Picked two 128 byte buffers: rbuffer, wbuffer. Based on the linked notes appeared the key takes 128. Used an array of k[0] to k[3] to check the key. Tean used (-) to decode + to encode. So used -32 for N. For stopping ASAP I added iFound as a means of signaling/ stopping. Threads check iFound state at the beginning of the thread loop and later sets iFound =1 (signaling stop) upon finding the correct key.

Initialized a new mutex “foundMutex” together with the ones from pt1. This protects iFound in the thread function (break special case there on the first). Added destroy for the new mutex, made sure to also have this on the return 1 error paths.



Issues: Misread, again. On pt1 Initially I modified prime.c. Assumed that it was ok to take away commented out txt brackets `/**/` + added the `<time.h>` library. (As not then optimize the actual function, would make it slower: not risk making it better). Reverted changes in prime.c to original state. In the EWA verification of pt1 I also had Makefile instead of makefile. (Had googled this and found Makefile was considered the most correct C89, but obviously both work).

Did not save valgrind file here on 4 but included this in the images.

Task 5. Documentation.

Considerations & execution: Building out step by step as recommended. The task I split into these source files: program5.c (main with the main order of operations). "task5_socket.c" takes care of socket creation, binding to 127.0.0.1, listening, and accepting the EWA client. "task5_handler.c" takes care of the protocol communication with EWA. And "task5_util.c" utility function (could have placed more here I think).

Handler grew rather large, in retrospect it might have been better splitting it into four source files and having something akin to a protocol only file. Otherwise, I tried to keep it to KISS and to stay a bit defensive.

Makefile I kept std 89, Wall and -g. Socket linker flag, it compiled without. Extensive debugging here, significantly larger project. Did not use the tiny debugger; relied on the compiler itself with -g and valgrind. Additionally utilized printf and puts to get & keep track of the current state. So not for every line, not to the same extent, more function by function or -function segment.

First step I only had the server ready to listen to socket and connect the client to it. Second step had it ready to handle helo, mail and rcpt to (but did not have the program ready to receive any files yet). Last step I added functionality to validate file name, receive files, write this file & handle quit command when called.

Did rerun valgrind a few times and saved the output file task5_valgrind.txt. Went with port 2525 to get around ports under 1024 needing root / privileged access.

The screenshot displays a CTF challenge environment. On the left, a terminal window titled 'cyber1@osboxes: ~/Work' shows the output of the 'Eastwill Assistant - EXAM 2026 v1' tool. The tool is running a SMTP server on port 2525. The terminal output shows the server successfully connecting to 127.0.0.1, receiving a HELO command, and then a MAIL FROM command. The server then receives a RCPT TO command and a DATA command. The server responds with '250 Message received' and '250 Ready for approved DATA file'. The client then sends a message body, and the server responds with '250 Data received'. Finally, the client sends a QUIT command, and the server responds with '250 closing connection'.

On the right, a file explorer window titled 'cyber1@osboxes: ~/Work/pg3401_exam26/task5_262' shows the contents of the task directory. The files listed are 'task5_handler.c', 'task5_socket.c', 'task5_util.c', 'task5_handler.h', 'task5_handler.c', 'program5.c', and 'task5_handler.c'. The 'task5_handler.c' file is selected, and its contents are displayed in the right pane. The code in 'task5_handler.c' shows a function 'SendAccept()' that formats a string and sends it to the client. The code also shows a warning about truncating the output of 'sprintf()'.

Issues: Significantly larger project, so a fair bit of debugging: typos, missing & wrongly placed symbols. I also introduced quite a lot of issues. From changing back and forth with naming conventions. Had “task5_” in many of them (but figured it took up a bit too much space). In the process of cutting those I was a bit too heavy handed (hit some of the ewpdef.h structs and defines that also used task5_).

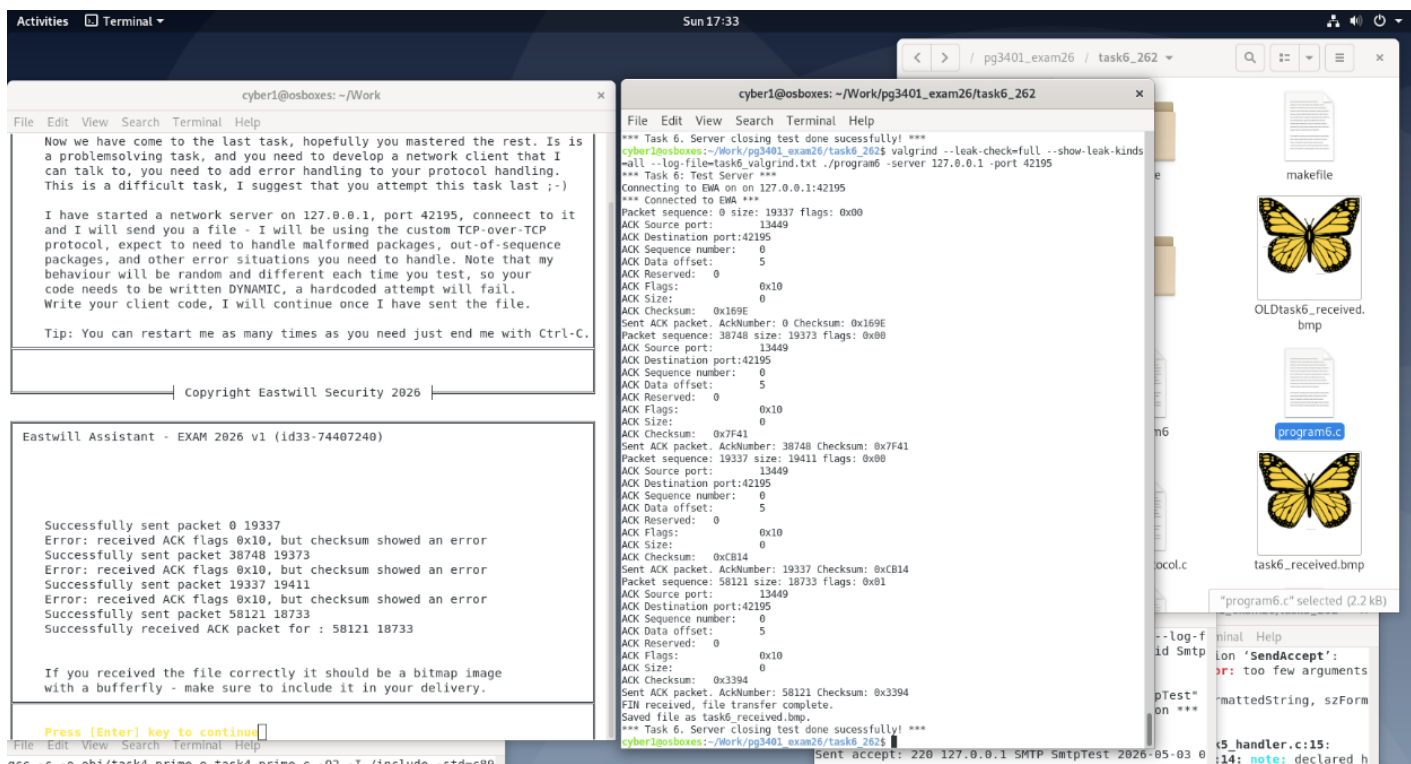
Longest snag though was with a flipped “>” so it would not begin listening. Compiler did not complain there. Had to work it out.

Task 6. Documentation.

Considerations & execution: I copied over large parts of task 5. Adapted these from a server to a client. So the program now connects to the EWA server using the IP address and port supplied on the command line. I split the code into program6.c, task6_socket.c, task6_protocol.c and task6_util.c. Task6_protocol, essentially the handler here (renamed from that), it did also grow a bit too large. Makefile I did not change notably from task 5. Just the usual flags Wall std89 and -g. And adding the new source files and headers.

Problem-solving: I used EWA feedback and printf debugging to inspect sequence numbers, flags, packet sizes and ACK/NACK behavior. Had several blocks in protocol.c where I printed out the header fields and checksum values. I tested the ACK number, flags and checksum handling until EWA accepted the ACKs. Another issue was that I initially got only about half of the butterfly image. That happened because packets could arrive out of sequence, and at first, I didn't account for that: just wrote packets directly to the BMP file.

Changed the code to store packets in a linked list sorted by sequence number (packet sequence ascending), then write them to task6_received.bmp in the correct order.



```
cyber1@osboxes: ~/Work
File Edit View Search Terminal Help
Now we have come to the last task, hopefully you mastered the rest. Is is
a problemsolving task, and you need to develop a network client that I
can talk to, you need to add error handling to your protocol handling.
This is a difficult task, I suggest that you attempt this task last ;-))

I have started a network server on 127.0.0.1, port 42195, connect to it
and I will send you a file - I will be using the custom TCP-over-TCP
protocol, expect to need to handle malformed packages, out-of-sequence
packages, and other error situations you need to handle. Note that my
behaviour will be random and different each time you test, so your
code needs to be written DYNAMIC, a hardcoded attempt will fail.
Write your client code, I will continue once I have sent the file.

Tip: You can restart me as many times as you need just end me with Ctrl-C.

Copyright Eastwill Security 2026

Eastwill Assistant - EXAM 2026 v1 (id33-74407240)

Successfully sent packet 0 19337
Error: received ACK flags 0x10, but checksum showed an error
Successfully sent packet 38748 19373
Error: received ACK flags 0x10, but checksum showed an error
Successfully sent packet 19337 19411
Error: received ACK flags 0x10, but checksum showed an error
Successfully sent packet 58121 18733
Successfully received ACK packet for : 58121 18733

If you received the file correctly it should be a bitmap image
with a butterfly - make sure to include it in your delivery.

Press [Enter] key to continue

cyber1@osboxes: ~/Work/pg3401_exam26/task6_262
File Edit View Search Terminal Help
*** Task 6. Server closing test done successfully! ***
cyber1@osboxes:~/Work/pg3401_exam26/task6_262$ valgrind --leak-check=full --show-leak-kinds=all --log-file=task6_valgrind.txt ./program6 -server 127.0.0.1 -port 42195
*** Task 6: Test Server ***
Connecting to EWA on 127.0.0.1:42195
*** Connected to EWA ***
Packet sequence: 0 size: 19337 flags: 0x00
ACK Source port: 13449
ACK Destination port: 42195
ACK Sequence number: 0
ACK Data offset: 5
ACK Reserved: 0
ACK Flags: 0x10
ACK Size: 0
ACK Checksum: 0x169E
Sent ACK packet. AckNumber: 0 Checksum: 0x169E
Packet sequence: 38748 size: 19373 flags: 0x00
ACK Source port: 13449
ACK Destination port: 42195
ACK Sequence number: 0
ACK Data offset: 5
ACK Reserved: 0
ACK Flags: 0x10
ACK Size: 0
ACK Checksum: 0x7F41
Sent ACK packet. AckNumber: 38748 Checksum: 0x7F41
Packet sequence: 19337 size: 19411 flags: 0x00
ACK Source port: 13449
ACK Destination port: 42195
ACK Sequence number: 0
ACK Data offset: 5
ACK Reserved: 0
ACK Flags: 0x10
ACK Size: 0
ACK Checksum: 0xCB14
Sent ACK packet. AckNumber: 19337 Checksum: 0xCB14
Packet sequence: 58121 size: 18733 flags: 0x01
ACK Source port: 13449
ACK Destination port: 42195
ACK Sequence number: 0
ACK Data offset: 5
ACK Reserved: 0
ACK Flags: 0x10
ACK Size: 0
ACK Checksum: 0x3394
Sent ACK packet. AckNumber: 58121 Checksum: 0x3394
FIN received, file transfer complete.
Saved file as task6_received.bmp.
*** Task 6. Server closing test done successfully! ***
cyber1@osboxes:~/Work/pg3401_exam26/task6_262$
```

Had half a butterfly, (the bottom half) for a while. I wanted to keep this in, thought it looked neat, but didn't change the name so managed to overwrite it. Still there are some differences with tiny artifacts in the images, so I kept the old full butterfly also.

Saved the valgrind file as task6_valgrind.txt.